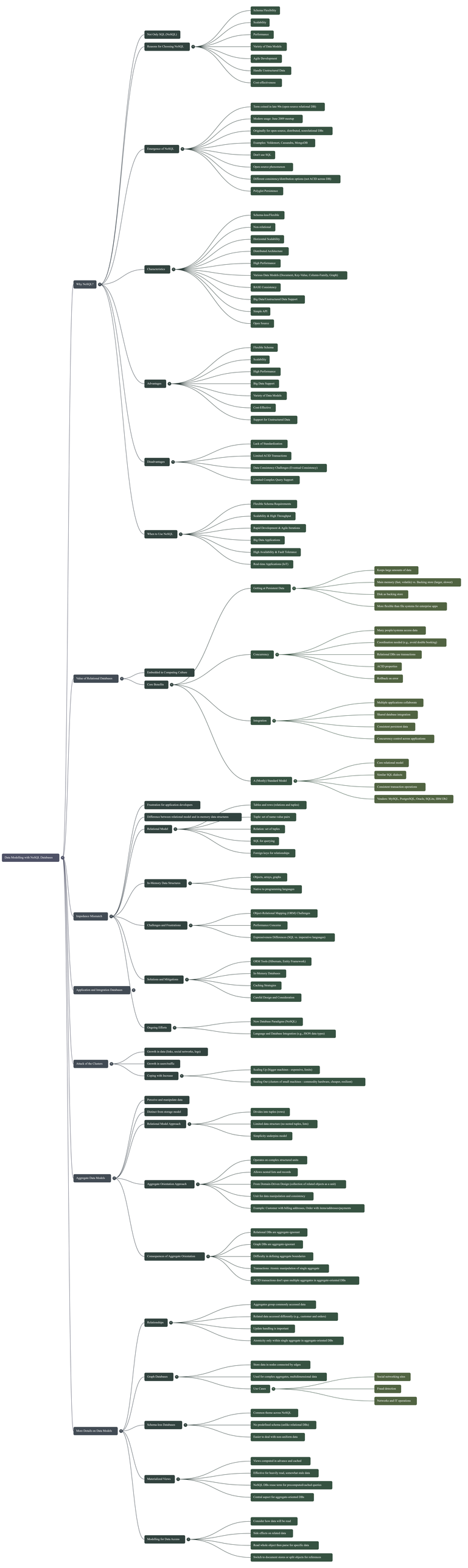
Data Modelling with NoSQL Databases
Why NoSQL?
Not Only SQL (NoSQL)
Reasons for Choosing NoSQL
Schema Flexibility
Scalability
Performance
Variety of Data Models
Agile Development
Handle Unstructured Data
Cost-effectiveness
Emergence of NoSQL
Term coined in late 90s (open-source relational DB)
Modern usage: June 2009 meetup
Originally for open-source, distributed, nonrelational DBs
Examples: Voldemort, Cassandra, MongoDB
Don't use SQL
Open-source phenomenon
Different consistency/distribution options (not ACID across DB)
Polyglot Persistence
Characteristics
Schema-less/Flexible
Non-relational
Horizontal Scalability
Distributed Architecture
High Performance
Various Data Models (Document, Key-Value, Column-Family, Graph)
BASE Consistency
Big Data/Unstructured Data Support
Simple API
Open Source
Advantages
Flexible Schema
Scalability
High Performance
Big Data Support
Variety of Data Models
Cost-Effective
Support for Unstructured Data
Disadvantages
Lack of Standardization
Limited ACID Transactions
Data Consistency Challenges (Eventual Consistency)
Limited Complex Query Support
When to Use NoSQL
Flexible Schema Requirements
Scalability & High Throughput
Rapid Development & Agile Iterations
Big Data Applications
High Availability & Fault Tolerance
Real-time Applications (IoT)
Value of Relational Databases
Embedded in Computing Culture
Core Benefits
Getting at Persistent Data
Keeps large amounts of data
Main memory (fast, volatile) vs. Backing store (larger, slower)
Disk as backing store
More flexible than file systems for enterprise apps
Concurrency
Many people/systems access data
Coordination needed (e.g., avoid double booking)
Relational DBs use transactions
ACID properties
Rollback on error
Integration
Multiple applications collaborate
Shared database integration
Consistent persistent data
Concurrency control across applications
A (Mostly) Standard Model
Core relational model
Similar SQL dialects
Consistent transaction operations
Vendors: MySQL, PostgreSQL, Oracle, SQLite, IBM Db2
Impedance Mismatch
Frustration for application developers
Difference between relational model and in-memory data structures
Relational Model
Tables and rows (relations and tuples)
Tuple: set of name-value pairs
Relation: set of tuples
SQL for querying
Foreign keys for relationships
In-Memory Data Structures
Objects, arrays, graphs
Native to programming languages
Challenges and Frustrations
Object-Relational Mapping (ORM) Challenges
Performance Concerns
Expressiveness Differences (SQL vs. imperative languages)
Solutions and Mitigations
ORM Tools (Hibernate, Entity Framework)
In-Memory Databases
Caching Strategies
Careful Design and Consideration
Application and Integration Databases
Ongoing Efforts
New Database Paradigms (NoSQL)
Language and Database Integration (e.g., JSON data types)
Attack of the Clusters
Growth in data (links, social networks, logs)
Growth in users/traffic
Coping with Increase
Scaling Up (bigger machines - expensive, limits)
Scaling Out (clusters of small machines - commodity hardware, cheaper, resilient)
Aggregate Data Models
Perceive and manipulate data
Distinct from storage model
Relational Model Approach
Divides into tuples (rows)
Limited data structure (no nested tuples, lists)
Simplicity underpins model
Aggregate Orientation Approach
Operates on complex structured units
Allows nested lists and records
From Domain Driven Design (collection of related objects as a unit)
Unit for data manipulation and consistency
Example: Customer with billing addresses, Order with items/addresses/payments
Consequences of Aggregate Orientation
Relational DBs are aggregate-ignorant
Graph DBs are aggregate-ignorant
Difficulty in defining aggregate boundaries
Transactions: Atomic manipulation of single aggregate
ACID transactions don't span multiple aggregates in aggregate-oriented DBs
More Details on Data Models
Relationships
Aggregates group commonly accessed data
Related data accessed differently (e.g., customer and orders)
Update handling is important
Atomicity only within single aggregate in aggregate-oriented DBs
Graph Databases
Store data in nodes connected by edges
Used for complex aggregates, multidimensional data
Use Cases
Social networking sites
Fraud detection
Networks and IT operations
Schema-less Databases
Common theme across NoSQL
No predefined schema (unlike relational DBs)
Easier to deal with non-uniform data
Materialized Views
Views computed in advance and cached
Effective for heavily read, somewhat stale data
NoSQL DBs reuse term for precomputed/cached queries
Central aspect for aggregate-oriented DBs
Modelling for Data Access
Consider how data will be read
Side effects on related data
Read whole object then parse for specific data
Switch to document stores or split objects for references